Reviewer Pack — Non-Commutative Fourier L1 Norm Distinguishes Read-Twice fro…

Entry 31bf8e50927c · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-10 11:23:10 UTC

Non-Commutative Fourier L1 Norm Distinguishes Read-Twice from Read-Once BPs

Entry ID: `31bf8e50927c` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-10 11:23:10 UTC

1. Conjecture

Field A (mathematical branch): Non-Commutative Harmonic Analysis **Field B** (complexity object): Read-Twice Branching Programs

Statement:

> For any read-twice branching program P over n variables, the L1 norm of its non-commutative Fourier transform satisfies $||F(P)||_1 \geq n/2$, whereas for read-once BPs, $||F(P)||_1 = O(\log n)$.

Rationale (proposer's reasoning):

> Non-commutative Fourier analysis captures branching program structure via group representations, exposing inherent symmetry gaps between read-twice and read-once models. The L1 norm quantifies this asymmetry through harmonic decomposition.

Taxonomy category: `FOURIER_ANALYTIC` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `977171fadc30f328`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.95	SAFE	UNCERTAIN
ALGEBRIZATION	SAFE	0.95	SAFE	UNCERTAIN

Barrier	Verdict	Confidence	LLM_1	LLM_2
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from typing import List, Dict

def generate_read_once_bp(n: int) -> List[List[int]]:
    bp = [[random.randint(0, 1) for _ in range(n)] for _ in range(n)]
    return bp

def generate_read_twice_bp(n: int) -> List[List[List[int]]]:
    bp = [[[random.randint(0, 1) for _ in range(n)] for _ in range(n)] for _ in range(n)]
    return bp

def discrete_fourier_transform(bp: List[List[int]]) -> List[List[complex]]:
    n = len(bp)
    omega = [math.exp(-2j * math.pi * i / n) for i in range(n)]
    F = [[0] * n for _ in range(n)]
    for k in range(n):
        for l in range(n):
            sum_val = 0
            for i in range(n):
                sum_val += bp[i][k] * omega[(i * l) % n]
            F[k][l] = sum_val / math.sqrt(n)
    return F

def non_commutative_fourier_norm(F: List[List[complex]]) -> float:
    norm = 0
    for row in F:
        for val in row:
            norm += abs(val)
    return norm

def run_trial(seed: int) -> Dict[str, any]:
    random.seed(seed)
    n = 40
    read_once_bp = generate_read_once_bp(n)
    read_twice_bp = generate_read_twice_bp(n)

    F_ro = discrete_fourier_transform(read_once_bp)
    F_rt = discrete_fourier_transform(read_twice_bp)
```

```
    norm_ro = non_commutative_fourier_norm(F_ro)
    norm_rt = non_commutative_fourier_norm(F_rt)

    metric_name = "non_commutative_fourier_norm"
    metric_value_ro = norm_ro
    metric_value_rt = norm_rt

    instances_tested = 2
    conjecture_holds = (norm_rt >= n / 2) and (norm_ro <= math.log(n))
    counterexample = "" if conjecture_holds else "read_once_bp" if norm_ro > n / 2 else "read_twic

    return {
        "metric_name": metric_name,
        "metric_value_ro": metric_value_ro,
        "metric_value_rt": metric_value_rt,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_ro = sum(r["metric_value_ro"] for r in results) / len(results)
    std_ro = math.sqrt(sum((r["metric_value_ro"] - mean_ro)**2 for r in results) / len(results))
    mean_rt = sum(r["metric_value_rt"] for r in results) / len(results)
    std_rt = math.sqrt(sum((r["metric_value_rt"] - mean_rt)**2 for r in results) / len(results))

    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean_ro={mean_ro} std_ro={std_ro} support_fraction={support_frac
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"read_once_bp\" first_failing_seed={first_failin
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_df9169c3.py", line 80, in <module>
    result = run_trial(seed)
```

```
            ^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_df9169c3.py", line 51, in run_trial
    F_ro = discrete_fourier_transform(read_once_bp)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_df9169c3.py", line 28, in discrete_four
    omega = [math.exp(-2j * math.pi * i / n) for i in range(n)]
             ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
TypeError: must be real number, not complex
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> Test crashed due to type error in Fourier transform implementation, preventing validation of conjecture | next: Fix the discrete_fourier_transform function to handle complex numbers properly and re-run tests

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	109467
2	propose	ollama_remote	qwen3:8b	0	0	89925
3	propose	ollama_remote	qwen3:8b	0	0	43490
4	preregistration	ollama_remote	qwen3:8b	0	0	24624
5	novelty	ollama_remote	qwen3:8b	0	0	23411
6	novelty	ollama_remote	qwen3:8b	0	0	11673
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14777
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10046
9	judge	ollama_remote	qwen3:8b	0	0	17660

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 345073 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/31bf8e50927c.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/31bf8e50927c.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/31bf8e50927c.tar.gz` (if generated)